

Ödev 21: Aksi Hâlde İçer Akaracağınız Çatıyı Kendiniz Kurun

Mikro çatıyı, bir framework'ün yaptığı şeyleri yapana kadar genişletin, sonra `assert` ifadesinin üretimde gizlediği hatayı bulun.

Sıfır üçüncü parti bağımlılık. Buradaki her şey yalnızca JDK.

Birinci Bölüm: Daha İyi Bir Çatı

`TestKit.java` dosyasını klasik biçimde oluşturun.

Şunları destekleyen bir `Suite` sınıfı yazın:

- `test(String name, Runnable body)` — bir geçme ya da başarısızlık kaydeder
- `assertEquals(expected, actual)` — başarısızlık ayrıntısı **her iki** değeri de göstermeli
- `assertThrows(Class<? extends Throwable>, Runnable)` — yalnızca o tam tip fırlatılırsa geçer

`assertThrows` metodunun bir değil **iki** başarısızlık durumu var. İkisini de farklı mesajlarla ele alın:

- Hiçbir şey fırlatılmadı
- Bir şey fırlatıldı, ama yanlış tip

Sonra en az şunları içeren bir suite çalıştırın: iki geçme, başarısız bir `assertEquals`, her iki `assertThrows` başarısızlık biçimi, ve beklenmedik bir şey fırlatan bir test.

Parametrelili testler ekleyin: bir döngüde üretilen dört satır girdiye karşı denetlenen tek bir davranış. Her durumu girdileriyle adlandırın, ve ona `case 3` demenin neden daha kötü olacağını bir yorumda söyleyin.

Bir özet satırı ve gerçek bir çatının döndüreceği çıkış kodunu yazdırın.

İkinci Bölüm: Yalıtım

Küçük, değiştirilebilir bir sınıf yazın, sonra ona karşı iki denetimi **iki kez** çalıştırın:

- Aralarında tek bir örnek paylaşarak
- Her birine kendi örneğini vererek

İkinci denetimin paylaşılan sürümde başarısız olduğunu, taze sürümde geçtiğini gösterin.

Açıklayın

Sıraya bağımlı bir testin neden **hiç test olmamasından daha kötü** olduğunu bir yorumda yazın.

Cevabınız sonradan ne olduğunu, kime olduğunu ve bir testin yokluğunun neden daha dürüst olduğunu

söylemeli.

Üçüncü Bölüm: `assert` İfadesinin Gizlediği Hata

İçinde şu bulunan bir metoda sahip bir sınıf yazın:

```
assert items.remove(item) : "expected " + item + " to be present";
```

Programınızı `-ea` **olmadan** çalıştırın ve koleksiyon boyutunu öncesinde ve sonrasında yazdırın. Sonra doğru sürümü yazın ve çalıştığını gösterin.

Açıklayın

Şunları kapsayan bir yorum:

- Bunun, onun için yazacağınız her testten neden geçtiği
- Üretimde ne olduğu, ve başarısızlığın neden bir çökme gibi görünmediği
- **Bunu önleyen kural**, tek cümlede

Ayrıca tespit numarasını kullanarak `assert`'lerin etkin olup olmadığını yazdırın, böylece çıktı her iki durumda da kendini açıklasın.

Dördüncü Bölüm: Hangi Denetim, Nereye

İki metot yazın:

1. Bir **public çağırının argümanını** doğrulayan biri
2. Kendi kodunuzun az önce kurduğu bir **iç değişmezi** öne süren biri

Birincisi için istisna, ikincisi için `assert` kullanın.

Açıklayın

Aralarında karar vermek için uygulayacağınız testi bir yorumda verin. Dal başına bir cümle, ve klavye başında gerçekten cevaplayabileceğiniz bir soru olmalı.

Kabul Kriterleri

- ☐ `java TestKit.java` ile çalışıyor, üçüncü parti import yok
- ☐ `Suite` sınıfı `test`, `assertEquals` ve `assertThrows` destekliyor
- ☐ `assertThrows` "hiçbir şey fırlatılmadı" ile "yanlış tip fırlatıldı" durumlarını ayırıyor
- ☐ Beklenmedik biçimde fırlatan bir test, çalıştırmanın çökmesi değil bir başarısızlık
- ☐ Parametrelili durumlar girdileriyle adlandırılmış

- ☐ Bir yorum `case 3` adının neden daha kötü olacağını söylüyor
- ☐ Özet sayıları, artı gerçek bir çatinın kullanacağı çıkış kodunu gösteriyor
- ☐ Paylaşılan durum ikinci denetimi başarısız kılıyor; taze durum geçiriyor
- ☐ Bir yorum sıraya bağımlı testlerin neden hiç testten kötü olduğunu açıklıyor
- ☐ Assert içine gizlenmiş silme, `-ea` olmadan **gerçekleşmediği** gösteriliyor
- ☐ Düzeltilmiş sürümün çalıştığı gösteriliyor
- ☐ Tek cümlelik kural belirtiliyor
- ☐ İstisna ve assert her biri ait olduğu yerde kullanılıyor, karar testi yazılmış

İleri Seviye

`test` metoduna bir zaman aşımı ekleyin, böylece asılı kalan bir gövde çalıştırmayı asmak yerine başarısızlık olarak bildirilsin.

Sonra bunu çalıştırmak için neyi devreye sokmak zorunda kaldığınızı ve bir zaman aşımının neden görüldüğünden daha zor olduğunu bir yorumda yazın. Bildirdikten sonra asılı testi çalıştıran iş parçacığına ne olduğunu düşünün.

İpucu, üçüncü bölümde silme gerçekleşiyorsa

Nasıl çalıştırdığınıza bakın. `-ea` ile assert çalışır ve silme işler, ki bu hatanın testten sağ çıkmasının tam sebebi. Üretimin çalıştığı gibi, bayraksız çalıştırın.